

MULTI-AGENT AI SYSTEM

心语日记

多智能体 AI 日记助手 · 项目说明文档



github.com/w020316/geren-riji

课程：大模型应用开发 | 日期：2026年5月

目 录

CONTENTS

- 01 一、项目概述
- 02 二、技术方案
- 03 三、系统架构
- 04 四、核心代码
- 05 五、功能模块详解
- 06 六、部署与演示

一、项目概述

1.1 项目背景

在快节奏的现代生活中，人们越来越需要一个能够倾听内心声音、记录生活点滴的智能伙伴。传统日记应用往往只是简单的文本记录工具，缺乏对用户情绪的感知和个性化内容的生成能力。本项目基于**多智能体协作架构 (Multi-Agent Architecture)**，打造了一款具有情绪感知能力、长期记忆管理和个性化日记生成的 AI 日记助手——「心语日记」。

1.2 项目目标

- 情绪感知**：通过大语言模型分析用户输入文本中的情绪状态，识别8种基本情绪类型及强度
- 记忆管理**：基于 ChromaDB 向量数据库实现语义级记忆存储与检索，支持长期记忆积累
- 日记生成**：结合情绪分析和历史记忆，自动生成温暖、个性化的日记内容
- 实时交互**：通过 SSE (Server-Sent Events) 技术实时推送多智能体协作进度，透明化 AI 工作过程
- 在线体验**：提供纯前端在线体验版，无需本地部署即可使用全部核心功能

1.3 技术栈总览

层级	技术选型	用途说明
大语言模型	DeepSeek API	情绪分析、信息提取、日记生成、对话回复
向量数据库	ChromaDB + BGE嵌入模型	长期记忆的向量化存储与语义检索
后端框架	FastAPI + uvicorn	RESTful API 服务、SSE 流式响应
前端技术	HTML/CSS/JavaScript	暗色主题交互界面、SSE 消息接收
数据持久化	JSON 文件 / localStorage	日记本地存储、浏览器端数据缓存
部署方式	Docker / GitHub Pages	容器化部署、静态站点托管

1.4 在线体验地址

 在线体验: <https://w020316.github.io/geren-riji/>

 纯前端运行，无需注册登录，打开即用

二、技术方案

2.1 多智能体协作架构设计

本项目的核心创新在于采用了**多智能体协作 (Multi-Agent Collaboration)** 的设计模式。不同于传统的单一 AI 调用方式，我们将日记生成的完整流程拆分为四个专业化的智能体角色，每个智能体负责特定的子任务：

情绪感知器 (EmotionSensor)

负责分析用户输入的情绪状态，识别情绪类型（开心/悲伤/焦虑等）和强度（1-10级），为后续处理提供情感基调。

记忆管家 (MemoryManager)

负责从对话中提取关键信息存入长期记忆库，并在需要进行语义检索，找到与当前话题相关的历史记忆。

日记生成器 (DiaryGenerator)

结合用户输入、情绪分析和相关记忆，调用大语言模型生成个性化、有温度的日记内容。

对话精灵 (DialogElf)

整合所有智能体的输出结果，生成最终的用户回复，协调情绪共情、日记呈现和记忆关联。

2.2 记忆系统的双层架构

为了兼顾记忆的深度和效率，本系统设计了**双层记忆架构**：

记忆类型	存储介质	容量	用途
短期记忆 (Short-Term Memory)	内存列表	最近 10 轮对话	提供上下文窗口，让 AI 理解当前对话的前因后果
长期记忆 (Long-Term Memory)	ChromaDB 向量数据库	无限制	语义级存储关键信息，支持跨会话的记忆检索与关联

长期记忆采用 **BGE 中文嵌入模型 (BAAI/bge-small-zh-v1.5)** 将文本转换为高维向量，使用余弦相似度进行语义匹配。这意味着即使查询词与存储的内容表面文字不同，只要语义相近就能被检索到。

2.3 SSE 实时进度推送

为了让用户直观地感受多智能体的工作过程，系统采用 **Server-Sent Events (SSE)** 技术实现实时进度推送。当用户发送消息后，前端依次接收到以下事件：

- 1. `emotion_start` → 情绪感知器开始工作
- 2. `emotion_done` → 情绪分析完成，附带情绪标签
- 3. `memory_start` → 记忆管家开始检索
- 4. `memory_done` → 记忆检索完成，返回相关记忆数量
- 5. `diary_start` → 日记生成器开始撰写
- 6. `diary_done` → 日记生成完成，返回日记全文
- 7. `response_start` → 对话精灵开始整合回复
- 8. `response_done` → 回复完成，返回完整回复内容

2.4 在线体验版的技术降级策略

由于 GitHub Pages 仅支持静态文件托管，无法运行 Python 后端服务，我们为在线体验版设计了**纯前端模拟方案**：

功能模块	完整版实现	体验版替代方案
情绪分析	DeepSeek LLM API	关键词规则引擎（20+ 关键词 × 8 种情绪）
记忆存储	ChromaDB 向量数据库	localStorage JSON 存储 + 关键词匹配搜索
日记生成	DeepSeek LLM API	模板化生成（3 种日记模板随机选择 + 变量填充）
对话回复	DeepSeek LLM API	按情绪分类的回复模板（每种情绪 2 个变体）
数据持久化	JSON 文件系统	localStorage 浏览器本地存储

三、系统架构

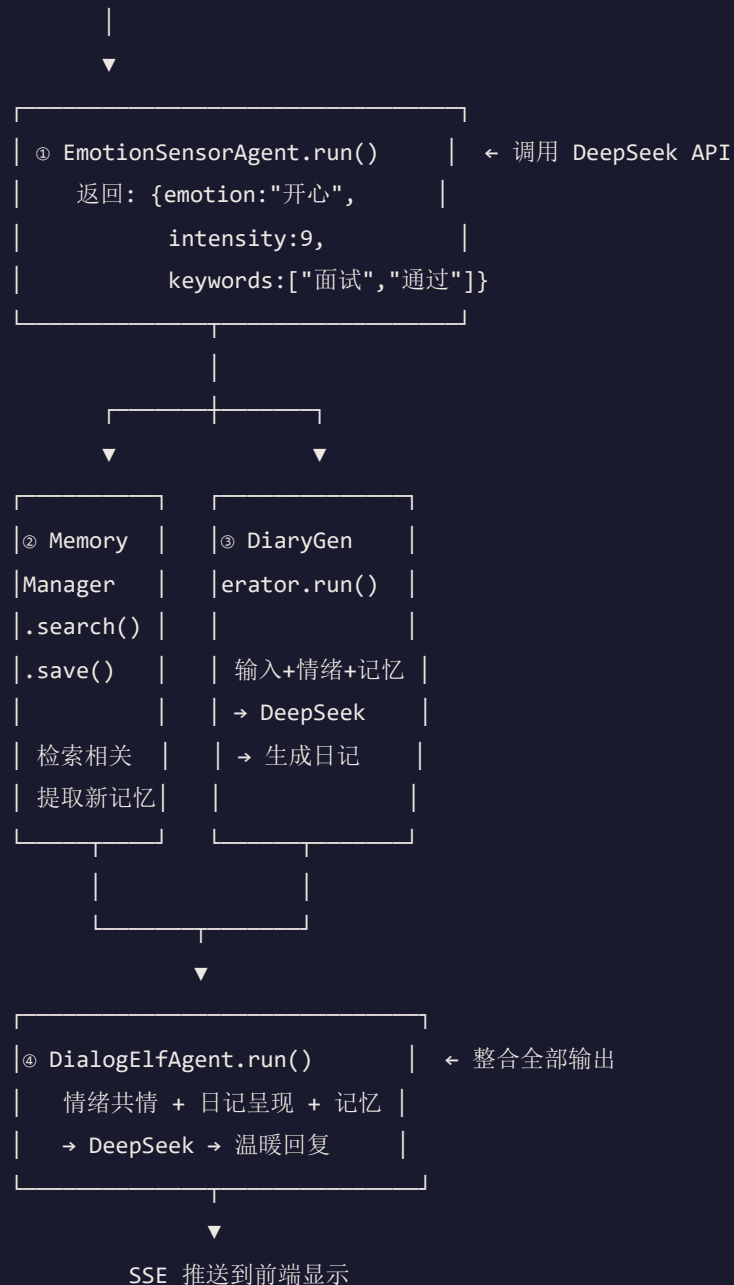
3.1 整体架构图



3.2 数据流图

// 用户发送消息后的完整数据流

用户输入文本 "今天面试通过了, 太开心了!"



3.3 项目目录结构

```
geren-riji/
├── main.py           # FastAPI 主入口, API 路由定义
├── config.py         # 配置管理 (API Key、路径等)
├── agents.py         # 四个智能体类 + 编排器
├── tools.py          # 工具函数 (情绪分析、日记生成)
├── memory.py         # 记忆系统 (ChromaDB + 短期记忆)
├── llm_client.py     # DeepSeek API 调用封装
├── diary_store.py    # 日记 JSON 文件持久化
├── static/
│   └── index.html    # 本地运行前端界面
├── docs/
│   ├── index.html    # GitHub Pages 在线体验版
│   └── architecture.svg # 架构图 SVG
├── Dockerfile        # 容器化部署配置
├── docker-compose.yml # Docker Compose 编排
├── requirements.txt   # Python 依赖
└── README.md         # 项目文档
```

四、核心代码

4.1 多智能体编排器 (agents.py)

编排器是整个系统的中枢，负责按顺序调度四个智能体并传递中间结果：

```

class MultiAgentOrchestrator:
    def __init__(self):
        self.memory_manager = MemoryManager()
        self.short_memory = ShortTermMemory(max_turns=SHORT_MEMORY_MAX_TURNS)
        self.emotion_agent = EmotionSensorAgent()
        self.memory_agent = MemoryManagerAgent(self.memory_manager)
        self.diary_agent = DiaryGeneratorAgent()
        self.dialog_agent = DialogElfAgent()

    def process(self, user_input: str) -> dict:
        # 1. 存入短期记忆
        self.short_memory.add("user", user_input)

        # 2. 情绪感知器分析情绪
        emotion_info = self.emotion_agent.run(user_input)

        # 3. 记忆管家：检索相关记忆 + 提取保存新记忆
        relevant_memories = self.memory_agent.search_relevant(user_input)
        self.memory_agent.extract_and_save(user_input, emotion_info)

        # 4. 日记生成器：结合输入、情绪、记忆生成日记
        diary = self.diary_agent.run(user_input, emotion_info, relevant_memories)

        # 5. 对话精灵：整合全部输出，生成温暖回复
        response = self.dialog_agent.run(
            user_input, emotion_info, diary, relevant_memories
        )

        # 6. 将回复存入短期记忆
        self.short_memory.add("assistant", response)

    return {
        "emotion": emotion_info,
        "diary": diary,
        "response": response,
        "memories_found": len(relevant_memories),
        "short_memory_context": self.short_memory.get_context()
    }

```

4.2 情绪分析工具函数 (tools.py)

```
def analyze_emotion(text: str) -> dict:
    system_prompt = """你是一个情绪分析专家。请分析用户输入文本中的情绪状态。
    返回JSON格式，包含以下字段：
    - emotion: 主要情绪（开心/悲伤/焦虑/愤怒/平静/惊喜/疲惫/感动）
    - intensity: 情绪强度（1-10，10为最强）
    - keywords: 情绪关键词列表
    - brief_analysis: 一句话情绪分析

    只返回JSON，不要其他内容。"""

    user_message = f"请分析以下文本的情绪：\n\n{text}"
    result = call_llm(system_prompt, user_message, temperature=0.3)
    return json.loads(result)
```

4.3 日记生成工具函数 (tools.py)

```
def generate_diary(user_input: str, emotion_info: dict, memories: list) -> str:
    today = datetime.now()
    today_str = today.strftime("%Y年%m月%d日")
    weekday_map = ["周一", "周二", ..., "周日"]
    weekday = weekday_map[today.weekday()]

    system_prompt = f"""你是一位温暖、细腻的日记撰写助手。
    重要规则：
    - 今天是{today_str}，{weekday}。必须使用这个真实日期，禁止编造其他日期
    - 以第一人称撰写
    - 融入情绪分析结果，让日记有情感深度
    - 适当引用历史记忆，体现连贯性
    - 语言优美但不做作，真实自然
    - 篇幅适中（200-500字）"""

    return call_llm(system_prompt, user_message, temperature=0.8)
```

4.4 ChromaDB 向量记忆管理 (memory.py)

```
class MemoryManager:
    def __init__(self):
        # 初始化 ChromaDB 持久化客户端
        self.client = chromadb.PersistentClient(path=CHROMA_DB_DIR)
        self.collection = self.client.get_or_create_collection(
            name="diary_memories",
            metadata={"hnsw:space": "cosine"} # 余弦相似度
        )
        # 加载 BGE 中文嵌入模型
        self.encoder = SentenceTransformer(EMBEDDING_MODEL)

    def save_memory(self, content: str, metadata: dict) -> str:
        # 文本向量化后存入 ChromaDB
        embedding = self._encode(content)
        memory_id = str(uuid.uuid4())
        self.collection.add(
            ids=[memory_id],
            embeddings=[embedding],
            documents=[content],
            metadatas=[metadata]
        )
        return memory_id

    def search_memory(self, query: str, n_results: int = 5) -> list:
        # 语义检索：查询文本向量化后在向量空间中找最相似的记录
        query_embedding = self._encode(query)
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=n_results
        )
        return results
```

4.5 FastAPI SSE 流式接口 (main.py 核心部分)

```
@app.post("/api/chat/stream")
async def chat_stream(req: ChatRequest):
    async def event_generator():
        orchestrator = MultiAgentOrchestrator()

        yield {"event": "emotion_start", "data": ""}
        emotion = orchestrator.emotion_agent.run(req.message)
        yield {"event": "emotion_done", "data": json.dumps(emotion)}

        yield {"event": "memory_start", "data": ""}
        memories = orchestrator.memory_agent.search_relevant(req.message)
        orchestrator.memory_agent.extract_and_save(req.message, emotion)
        yield {"event": "memory_done", "data": json.dumps({"count": len(memories)})}

        yield {"event": "diary_start", "data": ""}
        diary = orchestrator.diary_agent.run(req.message, emotion, memories)
        yield {"event": "diary_done", "data": diary}

        yield {"event": "response_start", "data": ""}
        response = orchestrator.dialog_agent.run(req.message, emotion, diary, memories)
        yield {"event": "response_done", "data": response}

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

五、功能模块详解

5.1 对话模块

对话是用户与系统交互的核心入口。用户在输入框中输入当天的想法、经历或心情，系统通过多智能体协作处理后返回包含情绪分析、生成日记和温暖回复的综合响应。

主要特性：

- SSE 实时流式推送，用户可看到每个智能体的工作状态（脉冲动画 → 完成 ✓）
- 快捷情绪标签（😊 开心 / 😫 疲惫 / 😟 焦虑 / 🌙 平静 / 😭 感动），一键快速输入
- 每日一句励志语随机展示，引导用户开始记录
- 打字机效果逐字显示 AI 回复，增强真实感
- 对话历史自动保存至 localStorage，刷新页面可恢复最近 50 条消息
- 多轮对话上下文：短期记忆保留最近 10 轮对话，让回复更连贯

5.2 日记本模块

日记本模块提供日记的浏览、搜索、编辑、删除和导出功能：

功能	说明
卡片浏览	以卡片网格形式展示所有日记，显示日期、情绪标签、预览内容
详情弹窗	点击卡片弹出详情，展示完整日记内容和原始输入
在线编辑	支持 contentEditable 实时编辑日记内容并保存
全文搜索	支持按日记内容、用户输入、情绪名称、日期实时过滤
Markdown 导出	支持单篇导出或全部批量导出为 Markdown 格式
数据导入导出	JSON 格式备份全部数据（日记+记忆），支持恢复

5.3 情绪统计模块

情绪统计模块帮助用户了解自身的情绪变化模式：

三大指标

总日记数、连续记录天数、最常出现的情绪类型

情绪分布图

8种情绪的彩色柱状图，直观展示各情绪占比

近7天趋势

柱状趋势图，悬停显示每天的情绪和强度详情

情绪日历

GitHub 风格热力图，每月每一天按情绪着色，支持月份切换

5.4 记忆管理模块

侧边栏记忆面板提供长期记忆的管理功能：

- **实时更新**：每次对话后自动提取关键信息并存入记忆库
- **语义搜索**：支持关键词过滤查找特定记忆
- **单条删除**：鼠标悬停显示删除按钮，可移除不需要的记忆
- **元数据显示**：每条记忆附带日期和关联情绪

六、部署与演示

6.1 本地部署

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置环境变量（复制模板并填入 API Key）
cp .env.example .env
# 编辑 .env 填写 DEEPSEEK_API_KEY

# 3. 启动服务
python main.py

# 4. 打开浏览器访问
# http://localhost:8000/static/index.html
```

6.2 Docker 部署

```
# 使用 docker-compose 一键启动
docker-compose up -d

# 访问 http://localhost:8000
```

6.3 GitHub Pages 在线体验

无需任何服务器，直接通过浏览器访问即可体验全部核心功能：

 在线地址：<https://w020316.github.io/geren-riji/>

 源码仓库：<https://github.com/w020316/geren-riji>

6.4 功能对比表

功能特性	完整版（本地/Docker）	体验版（GitHub Pages）
情绪分析	✔ DeepSeek LLM 深度理解	✔ 规则引擎（8种×20+词）
记忆系统	✔ ChromaDB 向量检索	✔ localStorage + 关键词匹配
日记生成	✔ LLM 动态创作	✔ 模板化生成（3种模板）
对话回复	✔ LLM 动态回复	✔ 模板化回复（每情绪2变体）
SSE 实时进度	✔ 后端推送	✔ 前端模拟延迟
日记 CRUD	✔ 完整支持	✔ 完整支持
情绪统计	✔ 完整支持	✔ 完整支持
数据持久化	✔ 文件系统	✔ localStorage
需要 API Key	✘ 是	✔ 否

心语日记 · 多智能体 AI 日记助手 | 课程作业项目 | 2026年5月

源码: <https://github.com/w020316/geren-riji>